

Python Automata Library (PAL) Cheat Sheet

import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode()** before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**.

Create state

pal.NewGrid(dims, dtype) — typed lattice

pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False) — individual agents; **dims=()** is nonspatial

pal.NewPopGrid(dims, capacity=None) — integer population counts

pal.NewPDEgrid(dims) — continuous field

pal.NewList() · **pal.NewMultinomial()** — integer query list · random-count sampler

Dimensions — 1–3 axes; a negative dimension wraps that axis, e.g. **(-nx, ny)**.

Shared lattice geometry

Properties — **xDim/yDim/zDim, nDims, wrapX/Y/Z**; a negative dimension enables wrapping on that axis.

Index conversion — **Tol(x[,y,z])**; **ltoX/Y/Z(i)**

Regions — **Box(x1,x2)** in 1D; **Box(x1,x2,y1,y2)** in 2D; add **(z1,z2)** in 3D. Each upper bound is exclusive: **[x1,x2)**, so **Box(0,4)** visits 0,1,2,3. Wrapped out-of-range coordinates wrap; nonwrapped ones are skipped.

Neighborhoods — **pal.MooreHood(dim, excludeCenter=False)** · **pal.VonNeumannHood(dim, excludeCenter=False)** · **pal.CircleHood(dim, rad, excludeCenter=False)**. These return relative offsets, not indices; pass them to **grid.Hood(hood, x[,y,z])**. Set **excludeCenter=True** to omit the center.

Grid / common indexing

Read/write — **g[i]** linear index; **g[x,y]** / **g[x,y,z]** coordinates; assign with **g[x,y]=v**. Slices return detached NumPy copies, so editing a slice does not update the grid.

Pattern — **g = pal.NewGrid((40,40), float)** · **g[10,12] = 1.0** · **v = g[10,12]**; supported dtypes include bool, fixed-width integers, and float32/float64.

Draw / output

Pixels — **pix, win = pal.StartPixWindow(xDim,yDim,scale=1,title='PAL',headless=False)**; set **pix[x,y]=RGB**; **win.Update()**; **win.Save(path, block=True)**; **win.Close()**.

GIF — **StartGif(path,delay=100)** · **AddGifFrame(block=False)** · **StopGif()**; call **Update()** before capture.

OpenGL — **pal.StartOpenGLWindow(...)**; draw with **Circle, Box, BoxSQ, Line, Borders**; scene controls include **Camera, Background, Clear**.

Lists + randomness

lList — **Append(i)**, **Clear()**, **Random()**, **Shuffle()**, indexing/**len**; **All()** detached copy; **Iter()** no-copy iteration.

RNG — **pal.Seed(seed)** · **pal.Random()** · **pal.RandInt(n)** → integer **0..n-1**; seed once for repeatable runs. PAL calls share a call-order-dependent random stream distinct from NumPy's RNG.

Multinomial — **m.Binomial(n,p)**; **Setup(...)** then **Sample(...)** for repeated multinomial draws.

AgentGrid

Create / move — **NewAgentSQ(x,y)** / **MoveSQ(a,x,y)** use lattice coordinates; **NewAgent(x,y)** / **Move(a,x,y)** use continuous positions. In 2D/3D, SQ forms take coordinates (or a linear index where supported); do not mix the two position systems.

Agent state — **grid[a,p]** reads/writes property **p**; **l(a)** is linear site index; **XSQ/YSQ/ZSQ(a)** are lattice coordinates; **X/Y/Z(a)** are continuous coordinates. Check **Alive(a)** before use when agents may be disposed; **Dispose(a)** removes an agent.

Queries — **GetPop()**, **AgentsAt(...)**, **LastAgent(...)**, **AgentsInRadius(...)**, **counts[...]**.

Iteration — **for a in agents.All():** ... iterates a detached snapshot and is safe for structural mutation such as **Dispose**. **AgentsInRadius(...)** returns nearby agents; wrapped displacements account for periodic boundaries.

Wrapping — **DispWrapX/Y/Z(p1,p2)** gives wrapped displacement.

PopGrid + PDEgrid

Transactional update — **Add(v, ...)** accumulates pending changes; **Update()** applies them together. **Reset()** clears both current and pending state. Direct **grid[...] = v** changes current state immediately, outside the transaction.

Population — **pop.GetPop()** total; **pop.All()** nonzero site indices. **capacity** limits total population.

PDE setup — **field.SetTimeSpaceStep(dt, dx[,dy,dz])** sets time and spatial steps before updates. Use spacing values that satisfy the selected scheme's stability requirements.

Diffusion — **Diffusion, DiffusionMask, DiffusionField, DiffusionInterfaces, DiffusionADI**; radial 1D: **DiffusionRadialCircle/Sphere**.

Advection — **Advection, AdvectionField, AdvectionInterfaces**.

More detail: [Manual](#) · [API Guide](#) · [API Reference \(Documentation/\)](#).